

P0517r0

Make future_error Constructible

Published Proposal, 10 November 2016

This version:

<http://wg21.link/P0517r0>

Author:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

Audience:

SG1, LEWG, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1	US 163
2	SG1 and LEWG Response to US 163
3	LWG Feedback
4	Proposed Wording
	References
	Informative References

This paper proposes a resolution to C++17 CD comment US 163, which requests the addition of a constructor to `future_error`. It is related to but does not overlap with the proposed resolution to [\[LWG2556\]](#), which is tentatively ready as of November 10th, 2016.

§ 1. US 163

The constructor for `future_error` should not be exposition only - this is the only exception class in the standard library that users have no clearly specified way to throw themselves. If we want the exception class to be limited to the standard library, at least make the exposition-only constructor private.

§ 2. SG1 and LEWG Response to US 163

SG1's tentative consensus was to make the constructor not-exposition-only and public. SG1 deferred to LEWG and LWG for the final determination with unanimous consent. LEWG forwarded this paper to LWG with unanimous consent.

§ 3. LWG Feedback

LWG made the following changes:

- Made the constructor `explicit`.

- Changed the constructor's parameter to be an `future_errc`.
- Added an exposition-only private `error_code` member to ease wording.

LWG took one straw poll:

Straw Poll: Make the `future_error` constructor's parameter an `future_errc`.

<i>F</i>	<i>N</i>	<i>A</i>
11	3	0

⇒ Consensus to make the constructor parameter a `future_errc`.

There was unanimous consent in LWG to move this paper at the Issaquah 2016 Friday plenary.

§ 4. Proposed Wording

The proposed changes are relative to [N4604], the Committee Draft for C++17.

Apply the following change to the class definition of `future_error` in 30.6.3 [futures.future_error]:

```
namespace std {
    class future_error : public logic_error {
    public:
        explicit future_error(error_code ecfuture_errc e); // exposition only

        const error_code& code() const noexcept;
        const char* what() const noexcept;
    private:
        error_code ec_; // exposition only
    };
}
```

Add the following member definition to 30.6.3 [futures.future_error] directly after the class definition of `future_error`:

```
explicit future_error(future_errc e);
```

Effects: Constructs an object of class `future_error` and initializes `ec_` with `make_error_code(e)`.

Apply the following change to the definition of `future_error::code()` in 30.6.3 [futures.future_error]:

```
const error_code& code() const noexcept;
```

Returns: ~~The value of `ec` that was passed to the object's constructor.~~ `ec_`.

§ References

§ Informative References

[LWG2556]

Agustín K-ballo Bergé. [Wide contract for future::share\(\)](http://cplusplus.github.io/LWG/lwg-active.html#2556). Tentatively Ready. URL: <http://cplusplus.github.io/LWG/lwg-active.html#2556>

[N4604]

